Análise de Dados

Licenciatura em Estatística

Rachid Muleia

UEM-Departamento de Matemática e Informática

2020/2021

1 Introdução ao ambiente R

2 Importação & Exportação de dados

3 Manipulação de dados

4 Análise exploratória de dados

5 Ciclos e Estruturas de control

6 Testes de Hipótese

7 Métodos avançados de leitura de dados

Introdução ao ambiente R

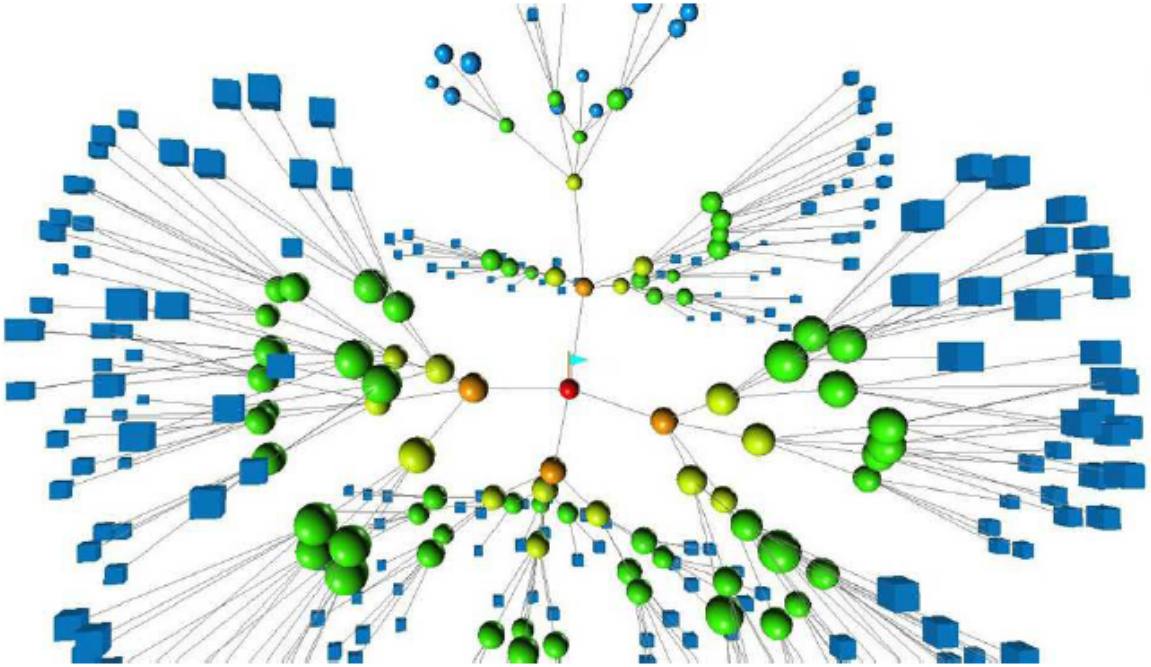
Análise de dados

O que é?

- Processo de limpeza, transformação e modelação de dados.
- Permite extrair informações úteis para toda de decisão.
- Na análise de dados, recolhemos e exibimos dados na forma de tabela, gráfico de barras, gráfico de linhas, etc.

Objectivo

Fazer uma análise aprofundada de um conjunto de dados volumoso e ser capaz de extrair informações úteis para tomada de decisão.



R Software

R sofware: programa gratuito para análise e visualização de dados.

- Linguagem de programação + Ferramenta de análise de dados
- Pode ser usado em várias áreas: saúde, banca, venda, etc.
- Analisar dados estruturados e não-estruturados.
- Desenvolvimento de applicativos webs, Shiny App

- Pode-se baixar o R software no repositório CRAN

  https://cran.r-project.org/bin/windows/base/R-4.0.1-win.exe

- O R software é usado em combinação com o Rstudio

- Rstudio é um editor, um ambiente de desenvolvimento integrado. https://rstudio.com/products/rstudio/download

- O processo de instalação do R software e Rstudio é semelhante à qualquer outro programa.

Download R

R-4.0.1 for Windows (32/64 bit)

Download R 4.0.1 for Windows (84 megabytes, 32/64 bit)

Installation and other instructions
New features in this version

If you want to double-check that the package you have downloaded matches the package distributed by CRAN, you can compare the md5sum of the .exe to the fingerprint on the master server. You will need a version of md5sum for windows: both graphical and command line versions are available.

Frequently asked questions

- Does R run under my version of Windows?
- How do I update packages in my previous version of R?
- Should I run 32-bit or 64-bit R?

Please see the R FAQ for general information about R and the R Windows FAQ for Windows-specific information.

Other builds

- Patches to this release are incorporated in the r-patched snapshot build.
- A build of the development version (which will eventually become the next major release of R) is available in the r-devel snapshot build.
- Previous releases

Note to webmasters: A stable link which will redirect to the current Windows binary release is <CRAN MIRROR>/bin/windows/base/release.html.

Last change: 2020-06-06

Objectivos da sessão

- Saber inicializar o R/Rstudio.
- Saber criar objectos em R.
- Conhecer os tipos de dados.
- Saber definir/criar vectores, matrizes, listas e data frames.
- Saber aceder os elementos de um vector, matriz, lista e data frame.

Rstudio - Inicialização

RStudio

File Edit Code View Plots Session Build Debug Profile Tools Help

Go to file/function | Addins | Project: (None)

Untitled1 | AD_DMI.Rmd

Source on Save | Run | Source

1

1:1 (Top Level) R Script

Console | R Markdown

C:/Users/lucp8943/Dropbox/Analise de Dados DMI/Slides Aulas/

```
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.

R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

> |
```

Environment | History | Connections

Import Dataset | List

Global Environment

Environment is empty

Files | Plots | Packages | Help | Viewer

Install | Update

Name	Description	Version
User Library		
abind	Combine Multidimensional Arrays	1.4-5
alphahull	Generalization of the Convex Hull of a Sample of Points in the Plane	2.2
askpass	Safe Password Entry for R, Git, and SSH	1.1
assertthat	Easy Pre and Post Assertions	0.2.1
automap	Automatic interpolation package	1.0-14
backports	Reimplementations of Functions Introduced Since R-3.0.0	1.1.5
base64enc	Tools for base64 encoding	0.1-3
bayestestR	Understand and Describe Bayesian Models and Posterior Distributions	0.4.0
BH	Boost C++ Header Files	1.69.0-1
BiocManager	Access the Bioconductor Project Package Repository	1.30.8
bit	A Class for Vectors of 1-Bit Booleans	1.1-14

R é uma calculadora científica

Operador	Descrição
+	Adição
-	Subtração
*	Multiplicação
/	Divisão
^	Exponenciação
%%	Modulos - resto da divisão
%/%	Inteiro da divisão

Pode efecturar operações matemáticas complexas:

- https://stat.ethz.ch/R-manual/R-devel/library/base/html/Arithmetic.html
- https://stat.ethz.ch/R-manual/R-devel/library/base/html/Trig.html

R é uma calculadora científica

```
> 1+2 # adição
[1] 3
> 2*4 # multiplicação
[1] 8
> 8/4 # divisão
[1] 2
> sqrt(4) # raiz quadrada
[1] 2
```

Tipo de Dados

- `numeric` - valores decimais ou inteiros.

```
>
> a <- 2
> a
[1] 2
> class(a)
[1] "numeric"
> b <- a + 0.1
> b
[1] 2.1
> class(b)
[1] "numeric"
```

Tipos de dados

- `character` - valores textuais.
- `logical` - valores lógicos (verdadeiro ou falso).

```
> caracter <- 'exportação'
> class (caracter)
[1] "character"
> logico <- TRUE
> class(logico)
[1] "logical"
```

Estrutura de dados

- `vector` (vector) - colecção de valores do mesmo tipo.
- `list` (listas) - colecção de estruturas de dados.
- `matrix` (matriz) - estrutura bidimensional rectangular.
- `data.frame` - colecção de vectores de mesma dimensão.

Vector

Estrutura unidimensional de colecção de valores do mesmo tipo.

No ambiente R, os vectores podem ser criados usando diferentes funções.

- `c()` - combina valores individuais.
- `seq` - cria uma sequência de valores.
- `rep()` - cria réplicas de valores

Vector - Exemplo

```
> # função c()
> c(2, 7, 8, 12, 3, 25)
[1]  2  7  8 12  3 25
> # pode atribuir o vector a uma variável, isto é, guardar.
> valores <- c(2, 7, 8, 12, 3, 25)
> # pode também criar um vector de caracteres
> frutas <- c('laranga','banana','pera','maçã')
> # função seq()
> seq(from=1, to=10)
 [1]  1  2  3  4  5  6  7  8  9 10
> seq(1, 10)
 [1]  1  2  3  4  5  6  7  8  9 10
> seq(1, 10,by=2)
[1] 1 3 5 7 9
> # função rep()
> rep(1,4)
[1] 1 1 1 1
> rep(4:6, 3)
[1] 4 5 6 4 5 6 4 5 6
> rep(1:4, each=2)
[1] 1 1 2 2 3 3 4 4
```

Operações com vectores

Pode aceder aos elemento de um vector usando o operador []

```
> x <- c(12, 15, 13, 17, 11)
> x[4]
[1] 17
> x[3:5]
[1] 13 17 11
> x[2]
[1] 15
> x[-(3:5)] # o sinal menos faz com que os elementos na posição 3:5 seja excluídos
[1] 12 15
```

Igualmente pode fazer operações entre vectores

```
> c(2, 5, 3) + c(4, 2, 7)
[1]  6  7 10
> 2 + c(2, 5, 3)
[1] 4 7 5
> c(2, 5, 3)^2
[1]  4 25  9
```

Operações com vectores

Algumas funções importantes:

- `length(x)` retorna o número total de elementos no vector.
- `x[length(x)]` retorna o último elemento do vector.
- `rev(x)` retorna o vector invertido.
- `sort(x)` retorna o vector ordenado.
- `unique(x)` retorna vetor sem múltiplos elementos.

Listas

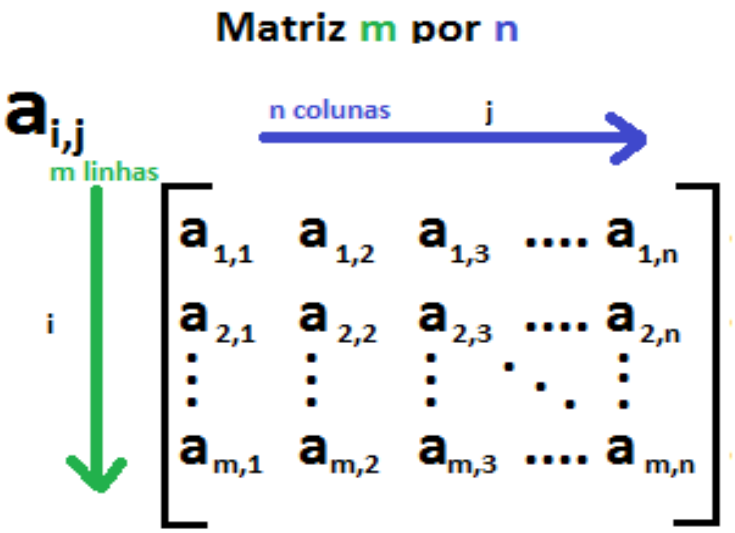
- Colecção de estruturas de dados.
- Uma lista pode conter qualquer tipo de dados, incluindo listas.
- Os objectos da lista podem ter tamanhos diferentes.
- Uma lista pode se criar usando a função `list()`

Lista - Exemplo

```
> minha_lista <- list(1:3, c("a", "b"), c(TRUE, FALSE, TRUE))
> str(minha_lista)
List of 3
 $ : int [1:3] 1 2 3
 $ : chr [1:2] "a" "b"
 $ : logi [1:3] TRUE FALSE TRUE
```

Matriz

Uma matriz é a organização de informações numéricas em uma tabela rectangular formada por linhas e colunas.



Matriz

No ambiente R, a matriz pode ser criada:

1. Usando a função `matrix()`.
2. Convertendo um vector para uma matriz.
3. Unindo dois ou mais vectors.

funções	Descrição
`matrix()`	Cria uma matriz especificando linhas e colunas
`dim()`	Retorna ou define a dimensão de um objecto
`cbind()`	Combina colunas
`rbind()`	Combina linhas

Matrizes - Exemplo

```
matrix(data = NA, nrow = 1, ncol = 1, byrow = FALSE,
       dimnames = NULL)
```

```
> m <- matrix(data=1:8,nrow=2, ncol=4)
> m
     [,1] [,2] [,3] [,4]
[1,]    1    3    5    7
[2,]    2    4    6    8
>
> m[1,4]
[1] 7
> m[1,1:3]
[1] 1 3 5
```

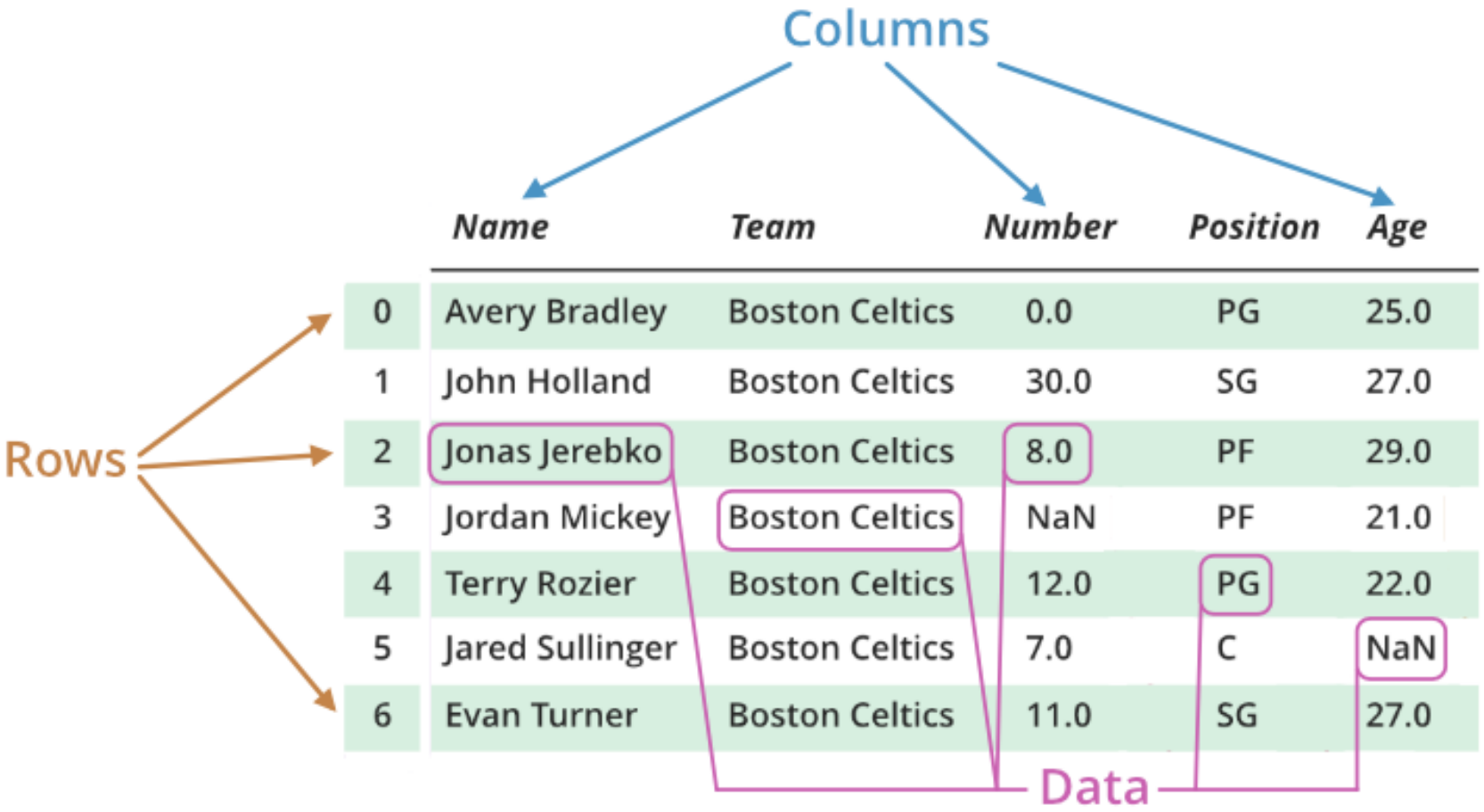
Matrizes- Exemplos

```
> cbind(1:3,5:7)
     [,1] [,2]
[1,]    1    5
[2,]    2    6
[3,]    3    7
> rbind(1:3,5:7,10:12)
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    5    6    7
[3,]   10   11   12
```

Data frames

- Colecção de vectores com igual tamanho.
- Estrutura bidimensional disposta em linhas e colunas.
  - **MAS**: DENTRO de uma coluna, cada célula deve ter o mesmo tipo de dados!
- `data frames` são usadas para representar todo conjunto de dados.
- As colunas contêm vectores com diferentes tipos de dados.

Data frame-layout



	Name	Team	Number	Position	Age
0	Avery Bradley	Boston Celtics	0.0	PG	25.0
1	John Holland	Boston Celtics	30.0	SG	27.0
2	Jonas Jerebko	Boston Celtics	8.0	PF	29.0
3	Jordan Mickey	Boston Celtics	NaN	PF	21.0
4	Terry Rozier	Boston Celtics	12.0	PG	22.0
5	Jared Sullinger	Boston Celtics	7.0	C	NaN
6	Evan Turner	Boston Celtics	11.0	SG	27.0

Uma data frame pode ser criada usando a função `data.frame()`.

Data frame - Exemplo

```
> df <- data.frame(ID = 1:3, Sexo = c("F", "F", "M"),
+ Peso = c(71, 60, 65))
>
> df <- data.frame(
+ ID = 1:3, # elementos da primeira coluna
+ Sexo = c("F", "F", "M"), # elementos da segunda coluna
+ Peso = c(71, 60, 65)) #   elementos da terceira coluna
```

Nota: as colunas devem ser do mesmo comprimento.

Data frame - Exemplo

```
> df <- data.frame(ID = 1:3, Sexo = c("F", "F", "M"),
+ Peso = c(71, 60, 65))
> df
  ID Sexo Peso
1  1    F   71
2  2    F   60
3  3    M   65
```

Data frame

- Criar uma `data frame` manualmente leva muito tempo.
- Além disso, a digitação convida erros de digitação.
- Deve evitar digitar grandes conjuntos de dados no R manualmente.
  - O melhor a fazer é importar todo conjunto de dados (base de dados).

As `data frames`, geralmente, são importadas usando as funções `read.csv()` e `read.table()`.

Detalhes na próxima sessão!!!

Importação & Exportação de dados

Objectivos da sessão

No final desta sessão deverá:

- Conhecer o formato ideal de uma `data frame`.
- Saber importar uma `data frame` para o R.
- Saber inspeccionar uma `data.frame`.
- Identificar anomalias na `data frame`.
- Exportar objectos em R para outro ambiente.

Passos para importação e exportação de `data frames`

1. Importar seus dados.
2. Inspeccionar, limpar e preparar os dados.
3. Fazer as análises.
4. Exportar os seus resultados
5. Limpar o ambiente R e fechar a sessão.

Como devem se apresentar os seus dados?

- As colunas devem representar variáveis.
- As linhas devem representar observações.
- Use a primeira linha para os nomes das variáveis.
- Todas as células em branco devem ser preenchidas com **NA**
- Armazene os dados no formato `.csv` e `.txt`, pois podem ser facilmente importados para o R.

NOTA IMPORTANTE: Todos os valores da mesma variável DEVEM estar na mesma coluna!

Exemplo: Tratamento para redução do peso.

Tratamento	Controlo	Placebo
65	70	80
70	80	86
55	90	78
68	100	98

Formato ideal de uma data frame.

Resposta	Método
65	Tratamento
70	Tratamento
55	Tratamento
68	Tratamento
70	Controlo
80	Controlo
90	Controlo
100	Controlo
80	Placebo
86	Placebo
78	Placebo
98	Placebo

Importação dados

Importe dados usando as funções `read.table()` ou `read.csv`

```
> meus_dados <- read.csv(file = 'MeusDados,txt')
> meus_dados <- read.csv(file = 'MeusDados.csv')
>
> # criam uma data frame meus_dados
```

Importação dados

Importe dados usando as funções `read.table()` ou `read.csv`

```
> meus_dados <- read.csv(file = 'MeusDados.csv')
> Error in file(file, "rt") : cannot open the connection
> In addition: Warning message:
> In file(file, "rt") :
>   cannot open file 'MeusDados.csv': No such file or directory
```

NOTA:

- Especificar o directório de trabalho usando a função `setwd()`.
- `setwd()` permite que o R saiba em que pasta está a base de dados.

Importação de dados - Argumentos importantes

Algumas dicas para reduzir possíveis erros na importação de dados.

- `header = TRUE` - informa ao R que a primeira linha da base de dados representa as variáveis.
- `sep = ','` informa ao R que os campos são separados por vírgulas.
- `strip.white = TRUE` remove o espaço em branco antes ou após caracteres que foram inseridos por engano durante a digitação de dados (EX: “pequeno” vs "pequeno ").
- `na.strings = ''` permite substituir células vazias por `NA`.

Importação de dados

Primeiro especifique o directório de trabalho

```
setwd('C:/Users/lucp8943/Dropbox/Analise de Dados DMI/Slides Aulas')
```

Agora vamos importar a base de dados `snail_feeding.csv`

usando a função `read.csv()`

```
> snail_data <- read.csv(file = "Snail_feeding.csv",
+ header = TRUE, strip.white = TRUE,
+ na.strings = " ")
```

Inspeccionando os dados

Após a importação dos dados, podemos usar as funções `str()`, `head()` e `tail()` para inspeccionar os dados.

```
> str(snail_data)
'data.frame':   769 obs. of  10 variables:
 $ Snail.ID: int  1 1 1 1 1 1 1 1 1 1 ...
 $ Sex     : chr  "male" "male" "males" "male" ...
 $ Size    : chr  "small" "small" "small" "small" ...
 $ Feeding : logi  FALSE FALSE FALSE FALSE FALSE TRUE ...
 $ Distance: num  0.17 0.87 0.22 0.13 0.36 0.84 0.69 0.6 0.85 0.59 ...
 $ Depth   : num  1.66 1.26 1.43 1.46 1.21 1.56 1.62 162 1.96 1.93 ...
 $ Temp    : int  21 21 18 19 21 21 20 20 19 19 ...
 $ X       : logi  NA NA NA NA NA NA ...
 $ X.1     : logi  NA NA NA NA NA NA ...
 $ X.2     : logi  NA NA NA NA NA NA ...
```

Inspeccionando os dados

- No slide anterior o execução do comando `str` mostra-nos que temos 10 variáveis, o que não é verdade.
- As colunas indesejadas podem ser excluídas.

```
> snail_data <- snail_data[ , 1:7]
> str(snail_data)
'data.frame':   769 obs. of  7 variables:
 $ Snail.ID: int  1 1 1 1 1 1 1 1 1 1 ...
 $ Sex     : chr  "male" "male" "males" "male" ...
 $ Size    : chr  "small" "small" "small" "small" ...
 $ Feeding : logi  FALSE FALSE FALSE FALSE FALSE TRUE ...
 $ Distance: num  0.17 0.87 0.22 0.13 0.36 0.84 0.69 0.6 0.85 0.59 ...
 $ Depth   : num  1.66 1.26 1.43 1.46 1.21 1.56 1.62 162 1.96 1.93 ...
 $ Temp    : int  21 21 18 19 21 21 20 20 19 19 ...
```

Inspeccionando os dados

A execução do comando `str` mostra-nos que a variável `Sex` tem 4 categorias. Esquisito !!!

```
> unique(snail_data$Sex)
[1] "male"   "males"  "Male"   "female"
```

Pode se também usar as seguintes funções para fazer uma breve inspecção nos dados:

- `summary()` : estatísticas sumárias para cada variável.
- `head()` : retorna as 6 primeiras linhas da base de dados.
- `tail()` : retorna as seis últimas linhas da base de dados.

Importação de dados armazenados na web

O R também pode ler dados armazenados numa página web.

```
> data_web = read.table("http://www.sthda.com/upload/decathlon.txt",
+                       header=TRUE, sep='\t',strip.white = TRUE)
> head(data_web)
     name X100m Long.jump Shot.put High.jump X400m X110m.hurdle Discus
1  SEBRLE 11.04      7.58    14.83      2.07 49.81        14.69  43.75
2    CLAY 10.76      7.40    14.26      1.86 49.37        14.05  50.72
3  KARPOV 11.02      7.30    14.77      2.04 48.37        14.09  48.95
4 BERNARD 11.02      7.23    14.25      1.92 48.93        14.99  40.87
5  YURKOV 11.34      7.09    15.19      2.10 50.42        15.31  46.26
6 WARNERS 11.11      7.60    14.31      1.98 48.68        14.23  41.10
  Pole.vault Javeline X1500m Rank Points Competition
1       5.02    63.19  291.7    1   8217           1
2       4.92    60.15  301.5    2   8122           1
3       4.92    50.31  300.2    3   8099           1
4       5.32    62.77  280.1    4   8067           1
5       4.72    63.44  276.4    5   8036           1
6       4.92    51.77  278.1    6   8030           1
```

Importação de dados armazenados na web

- Ler dados a partir da web não é uma boa prática.
- O aconselhável é baixar os dados e ler localmente.
- O R possui uma função que permite baixar os dados, a função `download.file()`

```
> download.file(url = "http://www.sthda.com/upload/decathlon.txt
+               destfile='decathlon')
```

- A função download pode baixar ficheiros com outras extensões.

Importação de dados no formato SPSS, SAS e STATA

- O R oferece recursos para ler ficheiros SPSS, SAS e STATA.
- Os recursos estão disponíveis na livraria `haven`. Como aceder??
- Precisamos instalar a livraria `haven` usando a função `install.packages()`.

Importação de dados no formato SPSS, SAS e STATA

- `read_sas()`: lê ficheiros `.sas7bdat` e `.sas7bcat` do SAS.
- `read_sav` ou `read_spss`: lê ficheiros `.sav` do SPSS.
- `read_dta`: lê ficheiros `.dta` do Stata.

```
> install.packages(haven)
> library(haven)
> dados_spss <- read_spss(file ='MZAR72FL.SAV')
```

Importação de dados no formato SPSS, SAS e STATA

Após ler os dados no formato spss, podemos aceder os nomes das variáveis.

```
> #install.packages(haven)
> library(haven)
> dados_spss <- read_spss(file ='MZAR72FL.SAV')
> attr(dados_spss$HIV03, 'label') # descrição da variável
[1] "Blood test result"
> attr(dados_spss$HIV03, 'labels') # labels dos valores assumido pela variável
                          HIV negative                          HIV  positive
                                     0                                     1
                         HIV2 positive                    HIV1 & HIV2 positive
                                     2                                     3
                    ERROR : V-, W+, M+                      ERROR : V-, W+, M-
                                     4                                     5
                    ERROR : V-, W-, M+                          Indeterminate
                                     6                                     7
Not enough samples to complete protocol                          Inconclusive
                                     8                                     9
```

Exportação de Dados

- O R permite exportar/gravar dados em vários formatos.
- Para exportação de dados usa-se a função `write.table()` ou `write.csv()`.

```
write.table(snail_data,  # Objecto que pretendemos exportar
            file='snail_inspecionado.txt', # nome do ficheiro
            sep=",", # separador de campos
           row.names=FALSE) # exlcuir os nomes das linhas.
```

- Para exportar dados no formato SPSS, SAS ou STATA usamos as funções, `write_sav()`, `write_sas()` e `write_dta()`, respectivamente.

Manipulação de dados

Objectivos

No final desta sessão, deverá saber:

- Organizar os dados no formato desejável;
- Criar novas variáveis;
- Filtrar linhas e seleccionar colunas;
- Juntar várias bases de dados numa só base (merge);
- Usar algumas funções da livraria `dplyr` para manipular dados;

Manipulação de dados

Para a manipulação de dados, iremos usar as funções da livraria `dplyr`.

- `select`: retornar um subconjunto das colunas de uma base de dados.
- `filter`: extrair um subconjunto de linhas de uma base de dados com base em condições lógicas.
- `arrange`: reordenar linhas de um quadro de dados.
- `rename`: renomeia as as variáveis de uma base de dados.
- `mutate`: adiciona novas variáveis / colunas ou transforma variáveis existentes.
- `summarise`/ `summarize`: gera estatísticas sumárias de diferentes variáveis na base de dados.
- `%>%` : operador `pipe` usado para ligar várias acções.

Operadores lógicos

O R disponibiliza os seguintes operadores lógicos:

Operador	Designção
!	Negação
|	Ou
&	E

Tabela de verdade-operador de negação !

x	!x	Example: x <- 5	Result
True	False	x > 5 !(x > 5)	TRUE FALSE
False	True	x < 5 !(x < 5)	FALSE TRUE

Tabela de verdade-operador &

x	y	x & y	Example: x <- 5 y <- 8	Result
False	False	False	(x > 5 & y > 8)	FALSE
False	True	False	(x > 5 & y < 8)	FALSE
True	False	False	(x < 5 & y > 8)	FALSE
True	True	True	(x < 5 & y < 8)	TRUE

Tabela de verdade-operador | (ou)

x	y	x | y	Example: x <- 5 y <- 8	Result
False	False	False	(x > 5 | y > 8)	False
False	True	True	(x > 5 | y < 8)	True
True	False	True	(x < 5 | y > 8)	True
True	True	True	(x < 5 | y < 8)	True

select

Para ilustrar o processo de manipulação de dados, vamos usar a base de dados `Chicago`.

```
> chicago <- readRDS("chicago.rds")
```

Antes de fazer qualquer operação, podemos inspeccionar base usando a função `str()` ou `dim()`.

```
> dim(chicago)
[1] 6940    8
> str(chicago)
'data.frame':   6940 obs. of  8 variables:
 $ city      : chr  "chic" "chic" "chic" "chic" ...
 $ tmpd      : num  31.5 33 33 29 32 40 34.5 29 26.5 32.5 ...
 $ dptp      : num  31.5 29.9 27.4 28.6 28.9 ...
 $ date      : Date, format: "1987-01-01" "1987-01-02" ...
 $ pm25tmean2: num  NA NA NA NA NA NA NA NA NA NA ...
 $ pm10tmean2: num  34 NA 34.2 47 NA ...
 $ o3tmean2  : num  4.25 3.3 3.33 4.38 4.75 ...
 $ no2tmean2 : num  20 23.2 23.8 30.4 30.3 ...
```

select

Suponha que se pretende seleccionar as 3 primeiras colunas.

```
> library(dplyr)
> names(chicago)[1:3]
[1] "city" "tmpd" "dptp"
> sel_col <- select(chicago, city:dptp)
> head(sel_col)
  city tmpd   dptp
1 chic 31.5 31.500
2 chic 33.0 29.875
3 chic 33.0 27.375
4 chic 29.0 28.625
5 chic 32.0 28.875
6 chic 40.0 35.125
```

select

Para alem de seleccionar, pode-se omitir as variáveis, bastando para tal usar o sinal menos `-`.

```
> select(chicago, -(city:dptp))
```

Este código indica que devemos incluir todas as variáveis excepto as variáveis `city` à `dptp`.

select

A função `select ()` também permite uma sintaxe especial que permite especificar nomes de variáveis com base em padrões.Por exemplo, se você quiser manter todas as variáveis que terminam com um "2", poderíamos fazer:

```
> sel_col <- select(chicago, ends_with('2'))
> str(sel_col)
'data.frame':   6940 obs. of  4 variables:
 $ pm25tmean2: num  NA NA NA NA NA NA NA NA NA NA ...
 $ pm10tmean2: num  34 NA 34.2 47 NA ...
 $ o3tmean2  : num  4.25 3.3 3.33 4.38 4.75 ...
 $ no2tmean2 : num  20 23.2 23.8 30.4 30.3 ...
```

Ou, se quiséssemos manter todas as variáveis que começam com um "d", poderíamos fazer:

```
> sel_col <- select(chicago, starts_with('d'))
> str(sel_col)
'data.frame':   6940 obs. of  2 variables:
 $ dptp: num  31.5 29.9 27.4 28.6 28.9 ...
 $ date: Date, format: "1987-01-01" "1987-01-02" ...
```

filter()

Extrair as linhas na base de dados`chicago` onde os níveis de poluição são maiores que 30.

```
> sel_fil <- filter(chicago,pm25tmean2 > 30)
> str(sel_fil)
'data.frame':   194 obs. of  8 variables:
 $ city      : chr  "chic" "chic" "chic" "chic" ...
 $ tmpd      : num  23 28 55 59 57 57 75 61 73 78 ...
 $ dptp      : num  21.9 25.8 51.3 53.7 52 56 65.8 59 60.3 67.1 ...
 $ date      : Date, format: "1998-01-17" "1998-01-23" ...
 $ pm25tmean2: num  38.1 34 39.4 35.4 33.3 ...
 $ pm10tmean2: num  32.5 38.7 34 28.5 35 ...
 $ o3tmean2  : num  3.18 1.75 10.79 14.3 20.66 ...
 $ no2tmean2 : num  25.3 29.4 25.3 31.4 26.8 ...
```

filter()

Podemos também estar interessados em filtrar todas as linhas que tem níveis de poluição maiores que 30 e temperatura maior que 80F.

```
> sel_fil <- filter(chicago,pm25tmean2 > 30 & tmpd > 80)
> str(sel_fil)
'data.frame':   17 obs. of  8 variables:
 $ city      : chr  "chic" "chic" "chic" "chic" ...
 $ tmpd      : num  81 81 82 84 85 84 82 82 81 82 ...
 $ dptp      : num  71.2 70.4 72.2 72.9 72.6 72.6 67.4 63.5 70.4 66.2 ...
 $ date      : Date, format: "1998-08-23" "1998-09-06" ...
 $ pm25tmean2: num  39.6 31.5 32.3 43.7 38.8 ...
 $ pm10tmean2: num  59 50.5 58.5 81.5 70 66 80.5 65 64 72.5 ...
 $ o3tmean2  : num  45.9 50.7 33 45.2 38 ...
 $ no2tmean2 : num  14.3 20.3 33.7 27.4 27.6 ...
```

arrange()

A função `arrange()` permite ordenar os dados segundo uma coluna/variável.

Por exemplo, podemos ordenar os dados em função da data

```
> chicago <- arrange(chicago, date)
> head(chicago, n=10)
   city tmpd   dptp       date pm25tmean2 pm10tmean2  o3tmean2 no2tmean2
1  chic 31.5 31.500 1987-01-01         NA   34.00000  4.250000  19.98810
2  chic 33.0 29.875 1987-01-02         NA         NA  3.304348  23.19099
3  chic 33.0 27.375 1987-01-03         NA   34.16667  3.333333  23.81548
4  chic 29.0 28.625 1987-01-04         NA   47.00000  4.375000  30.43452
5  chic 32.0 28.875 1987-01-05         NA         NA  4.750000  30.33333
6  chic 40.0 35.125 1987-01-06         NA   48.00000  5.833333  25.77233
7  chic 34.5 26.750 1987-01-07         NA   41.00000  9.291667  20.58171
8  chic 29.0 22.000 1987-01-08         NA   36.00000 11.291667  17.03723
9  chic 26.5 29.000 1987-01-09         NA   33.28571  4.500000  23.38889
10 chic 32.5 27.750 1987-01-10         NA         NA  4.958333  19.54167
```

arrange()

Para ordenar de forma decrescente é só fazer o uso da função `desc()`.

```
> chicago <- arrange(chicago, desc(date))
> head(chicago, n=10)
   city tmpd dptp       date pm25tmean2 pm10tmean2  o3tmean2 no2tmean2
1  chic   35 30.1 2005-12-31   15.00000       23.5  2.531250  13.25000
2  chic   36 31.0 2005-12-30   15.05714       19.2  3.034420  22.80556
3  chic   35 29.4 2005-12-29    7.45000       23.5  6.794837  19.97222
4  chic   37 34.5 2005-12-28   17.75000       27.5  3.260417  19.28563
5  chic   40 33.6 2005-12-27   23.56000       27.0  4.468750  23.50000
6  chic   35 29.6 2005-12-26    8.40000        8.5 14.041667  16.81944
7  chic   35 32.1 2005-12-25    6.70000        8.0 14.354167  13.79167
8  chic   37 35.2 2005-12-24   30.77143       25.2  1.770833  31.98611
9  chic   41 32.6 2005-12-23   32.90000       34.5  6.906250  29.08333
10 chic   22 23.3 2005-12-22   36.65000       42.5  5.385417  33.73026
```

rename()

A função `rename()` permite alterar o nome da coluna/variável.

Por exemplo, vamos alterar o nome `date` para `data` e `pm25tmean2` para `nivel_de_poluicao`.

```
> chicago <- rename(chicago, data = date,  nivel_de_poluicao = pm25tmean2)
> head(chicago, n=10)
   city tmpd dptp       data nivel_de_poluicao pm10tmean2  o3tmean2 no2tmean2
1  chic   35 30.1 2005-12-31          15.00000       23.5  2.531250  13.25000
2  chic   36 31.0 2005-12-30          15.05714       19.2  3.034420  22.80556
3  chic   35 29.4 2005-12-29           7.45000       23.5  6.794837  19.97222
4  chic   37 34.5 2005-12-28          17.75000       27.5  3.260417  19.28563
5  chic   40 33.6 2005-12-27          23.56000       27.0  4.468750  23.50000
6  chic   35 29.6 2005-12-26           8.40000        8.5 14.041667  16.81944
7  chic   35 32.1 2005-12-25           6.70000        8.0 14.354167  13.79167
8  chic   37 35.2 2005-12-24          30.77143       25.2  1.770833  31.98611
9  chic   41 32.6 2005-12-23          32.90000       34.5  6.906250  29.08333
10 chic   22 23.3 2005-12-22          36.65000       42.5  5.385417  33.73026
```

mutate()

A função `mutate()` existe para calcular transformações de variáveis em uma base de dados. Vamos converter a temperatura para graus celsius $(32^\circ F - 32) \times \frac{5}{9}$

```
> chicago <- mutate(chicago, temp_c = (tmpd-32)*5/9)
> head(chicago, n=10)
   city tmpd dptp       data nivel_de_poluicao pm10tmean2  o3tmean2 no2tmean2
1  chic   35 30.1 2005-12-31          15.00000       23.5  2.531250  13.25000
2  chic   36 31.0 2005-12-30          15.05714       19.2  3.034420  22.80556
3  chic   35 29.4 2005-12-29           7.45000       23.5  6.794837  19.97222
4  chic   37 34.5 2005-12-28          17.75000       27.5  3.260417  19.28563
5  chic   40 33.6 2005-12-27          23.56000       27.0  4.468750  23.50000
6  chic   35 29.6 2005-12-26           8.40000        8.5 14.041667  16.81944
7  chic   35 32.1 2005-12-25           6.70000        8.0 14.354167  13.79167
8  chic   37 35.2 2005-12-24          30.77143       25.2  1.770833  31.98611
9  chic   41 32.6 2005-12-23          32.90000       34.5  6.906250  29.08333
10 chic   22 23.3 2005-12-22          36.65000       42.5  5.385417  33.73026
      temp_c
1   1.666667
2   2.222222
3   1.666667
4   2.777778
```

mutate

A função `mutate()` permite criar uma variável sujeita a uma condição.

Por exemplo, crie uma variável níveis de poluição que assume valor 'baixo' poluição $<$ 30 e 'alto' caso contrário.

```
> chicago <- mutate(chicago,
+                   pol_ref = if_else(nivel_de_poluicao <30,'baixo','alto'))
> head(chicago)
  city tmpd dptp       data nivel_de_poluicao pm10tmean2  o3tmean2 no2tmean2
1 chic   35 30.1 2005-12-31          15.00000       23.5  2.531250  13.25000
2 chic   36 31.0 2005-12-30          15.05714       19.2  3.034420  22.80556
3 chic   35 29.4 2005-12-29           7.45000       23.5  6.794837  19.97222
4 chic   37 34.5 2005-12-28          17.75000       27.5  3.260417  19.28563
5 chic   40 33.6 2005-12-27          23.56000       27.0  4.468750  23.50000
6 chic   35 29.6 2005-12-26           8.40000        8.5 14.041667  16.81944
    temp_c pol_ref
1 1.666667   baixo
2 2.222222   baixo
3 1.666667   baixo
4 2.777778   baixo
5 4.444444   baixo
```

group_by()

A função group_by() é usada para gerar estatísticas sumárias nos estratos/grupos definidos por uma variável. Por exemplo, neste conjunto de dados de poluição do ar, você pode querer saber qual é o nível médio anual de PM2.5. Então o estrato é o ano.

```
> library(lubridate)
> chicago <- mutate(chicago, ano = year(data))
> anos <- group_by(chicago,ano )
> summarise(anos, pm25 = mean(nivel_de_poluicao, na.rm = TRUE),
+           o3 = max(o3tmean2, na.rm = TRUE),
+           no2 = median(no2tmean2, na.rm = TRUE))
# A tibble: 19 x 4
     ano  pm25    o3   no2
   <dbl> <dbl> <dbl> <dbl>
 1  1987 NaN    63.0  23.5
 2  1988 NaN    61.7  24.5
 3  1989 NaN    59.7  26.1
 4  1990 NaN    52.2  22.6
 5  1991 NaN    63.1  21.4
 6  1992 NaN    50.8  24.8
 7  1993 NaN    44.3  25.8
 8  1994 NaN    52.2  28.5
```

group_by() , summarise() e %>%

Para mostrar mais aplicação destas duas funções, vamos usar a base de dados `iris`

- calcular o comprimento médio, máximo e mínimo das pétalas por espécie.

```
> data("iris")
> str(iris)
'data.frame':   150 obs. of  5 variables:
 $ Sepal.Length: num  5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
 $ Sepal.Width : num  3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
 $ Petal.Length: num  1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
 $ Petal.Width : num  0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
 $ Species     : Factor w/ 3 levels "setosa","versicolor",..: 1 1 1 1 1 1 1 1 1 1 ...
```

group_by() , summarise() e %>%

```
> iris %>% group_by(Species) %>%
+   summarize(mean(Petal.Length), max(Petal.Length), min(Petal.Length))
# A tibble: 3 x 4
  Species    `mean(Petal.Length)` `max(Petal.Length)` `min(Petal.Length)`
  <fct>                     <dbl>               <dbl>               <dbl>
1 setosa                     1.46                 1.9                 1
2 versicolor                 4.26                 5.1                 3
3 virginica                  5.55                 6.9                 4.5
```

Exercícios - tente resolver, vai ajudar !!!

1) Quantos agregados têm acesso a electricidade?

2) Quantos vivem em casas modernas: - Uma casa é moderna se: -Chão: adobe, parquet, cimento, mosaico - Paredes: cimento, blocos, madeira - Tecto: Metal/zinco, madeira, telhas,

3) Transforme a idade registada em anos para idade em meses.

4) A dimensão da rede foi registada em metros quadrados, converta os valores para centímetros quadrados.

5) Seleccione todos indivíduos que professam a religião católica ou islámica.

6) Seleccione indivíduos que professam religião católica, cujo principal tipo de pesca seja arrasto ou rede de emalhar.

7) Determine o número de pescadores por cada tipo de pesca.

8) Calcule o número de respondentes por cada método de conservação e tipo de pesca.

Junção de base de dados

Para a combinação de dados iremos usar as seguintes funções:

- `rbind()` para juntar linhas;
- `cbind()` para juntar colunas;
- `merge()` para juntar bases de dados com base numa chave de identificação;

rbind()

A função `rbind()` retorna a junção de linhas de duas bases de dados, que têm mesmas variáveis (colunas).

Name	City	Country
Lenna	San Francisco	US
Malcom	New York	US
Akiko	Tokyo	Japan

+

Name	City	Country
Lenna	San Francisco	US
Thomas	London	UK
Diane	Chicago	US

Name	City	Country
Lenna	San Francisco	US
Malcom	New York	US
Akiko	Tokyo	Japan
Lenna	San Francisco	US
Thomas	London	UK
Diane	Chicago	US

cbind()

Retorna a junção de colunas de duas bases de dados, quando estas têm o mesmo número de linhas.

ID	var1	var2	var3
588	2	d	1
654	1	y	1
527	1	o	0
955	2	c	0
954	1	t	0

ID	var1	var2	var3
588	290	Apples	Breakfast
654	81	Bananas	Snack
527	63	Apples	Snack
955	6	Pears	Snack
954	146	Pears	Breakfast

ID	var1	var2	var3	var4	var5	var6
588	2	d	1	225	Apples	Breakfast
654	1	y	1	56	Bananas	Snack
527	1	o	0	245	Apples	Snack
955	2	c	0	46	Pears	Snack
954	1	t	0	121	Pears	Breakfast

cbind()

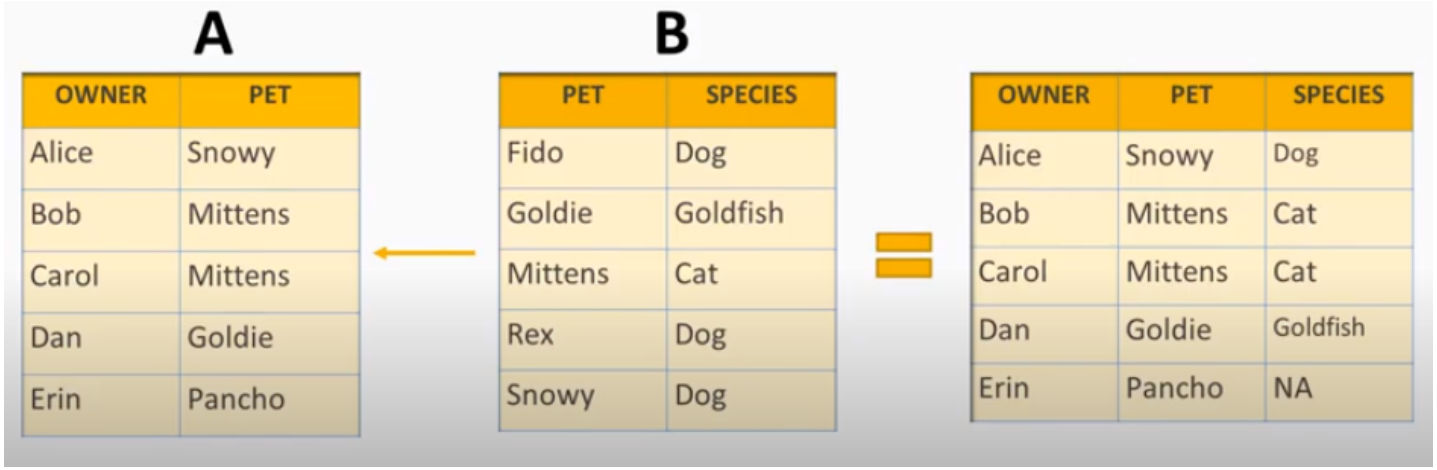
```
> data_1 <- data.frame(ID=c(588,654,527,955,954), # primeira coluna
+                      var1=c(2,1,1,2,1),         # segunda coluna
+                      var2=c('d','y','o','c','t'))# terceira coluna
> data_2 <- data.frame(ID=c(588,654,527,955,954),
+                      var3=c(290,81,63,6,146),
+                    var4=c('Apples','Bananas','Apples','Pears','Pears'),
+                    var5=c('Breakfast','Snack','Snack','Snack','Breakfast'))
> data_3 <- cbind(data_1,data_2)
> data_3
   ID var1 var2  ID var3    var4      var5
1 588    2    d 588  290  Apples Breakfast
2 654    1    y 654   81 Bananas     Snack
3 527    1    o 527   63  Apples     Snack
4 955    2    c 955    6   Pears     Snack
5 954    1    t 954  146   Pears Breakfast
```

Merge/Join

- Merge permite unir duas bases dados com base numa identificação(chave-ID).
- Existem vários tipos de merge:
  - left join
  - right outer
  - inner join
  - full join

Left join

left join - retorna todos elementos comuns nas duas tabelas (base de dados) e os restantes elementos na tabela a esquerda.



A

OWNER	PET
Alice	Snowy
Bob	Mittens
Carol	Mittens
Dan	Goldie
Erin	Pancho

B

PET	SPECIES
Fido	Dog
Goldie	Goldfish
Mittens	Cat
Rex	Dog
Snowy	Dog

OWNER	PET	SPECIES
Alice	Snowy	Dog
Bob	Mittens	Cat
Carol	Mittens	Cat
Dan	Goldie	Goldfish
Erin	Pancho	NA

Left join

```
> data_1 <- data.frame(owner = c('Alice', 'Bob','Carol','Dan','Erin'),
+                      pet = c('Snowy','Mittens','Mittens', 'Goldie','Pancho'))
> data_2 <- data.frame(pet=c('Fido','Goldie','Mittens','Rex','Snowy'),
+                      species =c('Dog','Goldfish','Cat','Dog','Dog'))
> data_3 = merge(data_1,data_2, by.x='pet', by.y='pet', all.x=TRUE)
> data_3
      pet owner  species
1  Goldie   Dan Goldfish
2 Mittens   Bob      Cat
3 Mittens Carol      Cat
4  Pancho  Erin     <NA>
5   Snowy Alice      Dog
```

Right join

right join- retorna todas as linhas comum nas duas tabelas e todas as restantes linhas da tabela a direita.

A

OWNER	PET
Alice	Snowy
Bob	Mittens
Carol	Mittens
Dan	Goldie
Erin	Pancho

B

PET	SPECIES
Fido	Dog
Goldie	Goldfish
Mittens	Cat
Rex	Dog
Snowy	Dog

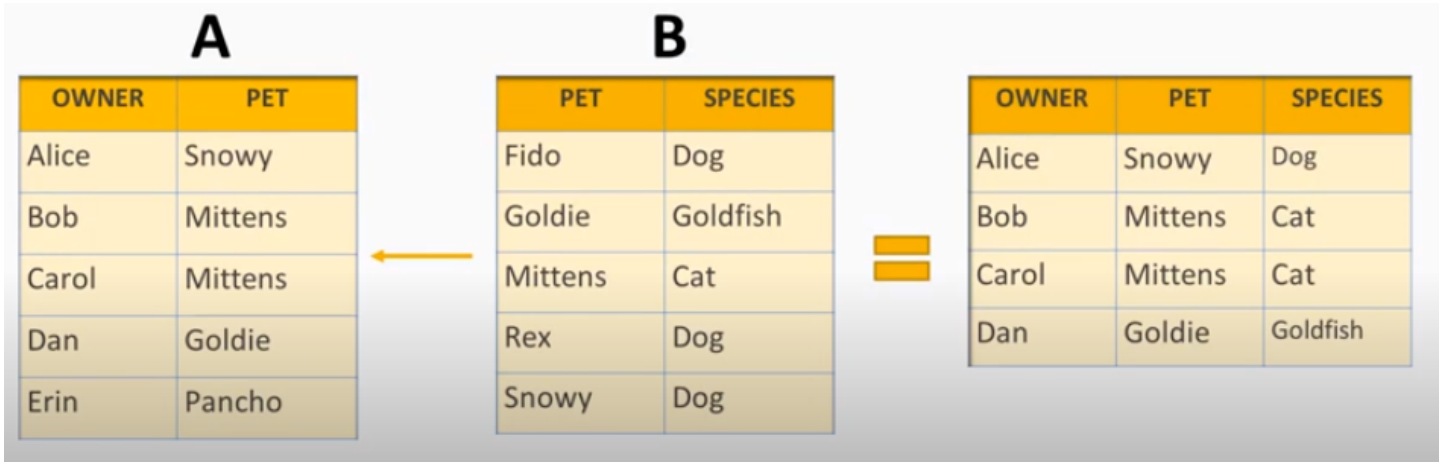
OWNER	PET	SPECIES
Alice	Snowy	Dog
Bob	Mittens	Cat
Carol	Mittens	Cat
Dan	Goldie	Goldfish
Erin	Pancho	NA

Right join

```
> data_1 <- data.frame(owner = c('Alice', 'Bob','Carol','Dan','Erin'),
+                      pet = c('Snowy','Mittens','Mittens', 'Goldie','Pancho'))
> data_2 <- data.frame(pet=c('Fido','Goldie','Mittens','Rex','Snowy'),
+                      species =c('Dog','Goldfish','Cat','Dog','Dog'))
> data_3 = merge(data_1,data_2, by.x='pet', by.y='pet', all.y=TRUE)
> data_3
      pet owner  species
1    Fido  <NA>      Dog
2  Goldie   Dan Goldfish
3 Mittens   Bob      Cat
4 Mittens Carol      Cat
5     Rex  <NA>      Dog
6   Snowy Alice      Dog
```

Natural join

natural join retorna todas as linhas comuns nas duas tabelas (base de dados).



A

OWNER	PET
Alice	Snowy
Bob	Mittens
Carol	Mittens
Dan	Goldie
Erin	Pancho

B

PET	SPECIES
Fido	Dog
Goldie	Goldfish
Mittens	Cat
Rex	Dog
Snowy	Dog

OWNER	PET	SPECIES
Alice	Snowy	Dog
Bob	Mittens	Cat
Carol	Mittens	Cat
Dan	Goldie	Goldfish

Natural join

```
> data_1 <- data.frame(owner = c('Alice', 'Bob','Carol','Dan','Erin'),
+                      pet = c('Snowy','Mittens','Mittens', 'Goldie','Pancho'))
> data_2 <- data.frame(pet=c('Fido','Goldie','Mittens','Rex','Snowy'),
+                      species =c('Dog','Goldfish','Cat','Dog','Dog'))
> data_3 = merge(data_1,data_2, by.x='pet', by.y='pet')
> data_3
      pet owner  species
1  Goldie   Dan Goldfish
2 Mittens   Bob      Cat
3 Mittens Carol      Cat
4   Snowy Alice      Dog
```

Full join

full join - retorna todas as linhas das tabelas unidas sejam elas correspondentes ou não.

A

OWNER	PET
Alice	Snowy
Bob	Mittens
Carol	Mittens
Dan	Goldie
Erin	Pancho

B

PET	SPECIES
Fido	Dog
Goldie	Goldfish
Mittens	Cat
Rex	Dog
Snowy	Dog

OWNER	PET	SPECIES
Alice	Snowy	Dog
Bob	Mittens	Cat
Carol	Mittens	Cat
Dan	Goldie	Goldfish
Erin	Pancho	NA
NA	Fido	Dog
NA	Rex	Dog

Full Join

```
> data_1 <- data.frame(owner = c('Alice', 'Bob','Carol','Dan','Erin'),
+                      pet = c('Snowy','Mittens','Mittens', 'Goldie','Pancho'))
> data_2 <- data.frame(pet=c('Fido','Goldie','Mittens','Rex','Snowy'),
+                      species =c('Dog','Goldfish','Cat','Dog','Dog'))
> data_3 = merge(data_1,data_2, by.x='pet', by.y='pet', all=TRUE)
> data_3
      pet owner  species
1    Fido  <NA>      Dog
2  Goldie   Dan Goldfish
3 Mittens   Bob      Cat
4 Mittens Carol      Cat
5  Pancho  Erin     <NA>
6     Rex  <NA>      Dog
7   Snowy Alice      Dog
```

Análise exploratória de dados

Objectivos

No final desta sessão, deverá saber:

- Identificar os diferentes tipos de variáveis.
- Sumarizar os dados usando estatísticas descritivas.
- Visualizar os dados através de gráficos.
- Conduzir um teste de associação de Qui-quadrado.
- Saber conduzir um teste t e uma ANOVA.

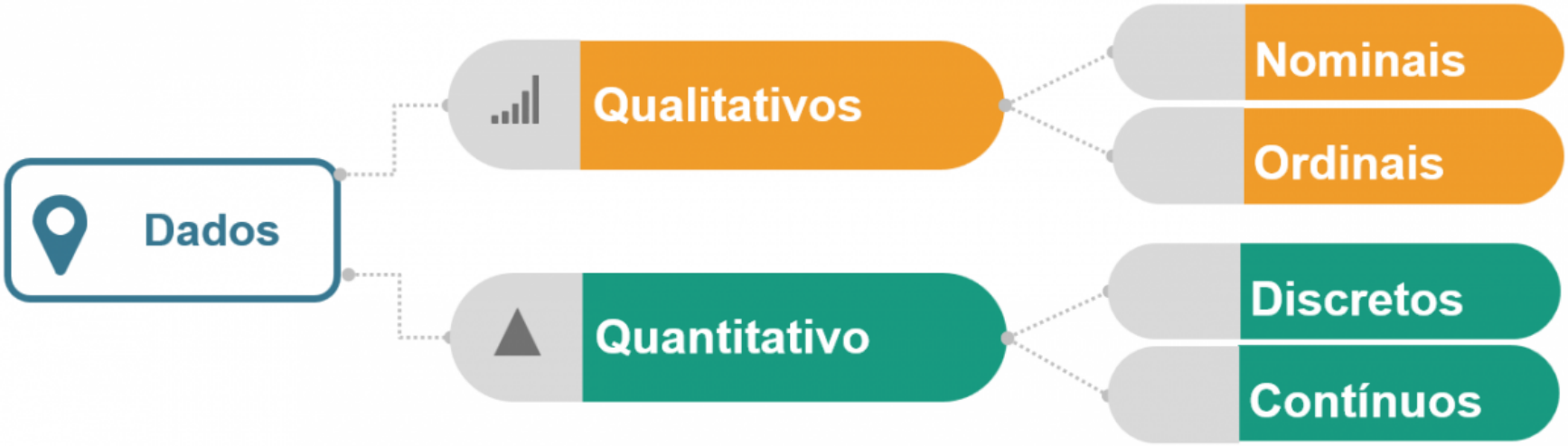
Análise exploratória de dados (AED)

Na análise estatística, a AED é um dos primeiros passos antes de avançar com análises mais complexas.

Porquê?

- Identificar anomalias nos dados.
- Familiarizarmo-nos mais com os dados.
- Descobrir padrões e tendências.
- Formular hipóteses.

Tipo de dados/variáveis



- Quantitativo: altura, peso, temperatura, idade,...
- Qualitativo: cor dos olhos, nome, classificação numa competição,...

Medidas sumárias/ descritivas

Para variáveis quantitativas/númericas, pode-se resumir os dados usando as seguintes funções:

- Medidas de tendência central- Média, Mediana e Moda.
- Medidas de Posição: Quartis, Decis e Percentis,. . .
- Medidas de Dispersão: Variância, Desvio-Padrão, Coeficiente de variação:

Medidas sumárias/ descritivas

O R possui várias funcionalidades para sumarizar os dados.

- `summary()`: retorna várias medidas descritivas.
- `mean()` e `median()`: retorna a média e a mediana, respectivamente.
- `quantile()`: retorna quartis, decis ou percentis.
- `var()` e `sd()`: retorna a variância e o desvio-padrão, respectivamente.

Medidas sumárias/ descritivas

Vamos usar a base de dados `iris` para ilustrar o cálculo de medidas sumárias no R.

```
> data('iris') # chama uma base interna do R.
> str(iris) # mostra a estrutura da base de dados.
'data.frame':   150 obs. of  5 variables:
 $ Sepal.Length: num  5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
 $ Sepal.Width : num  3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
 $ Petal.Length: num  1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
 $ Petal.Width : num  0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
 $ Species     : Factor w/ 3 levels "setosa","versicolor",..: 1 1 1 1 1 1 1 1 1 1 ...
> summary(iris)
  Sepal.Length    Sepal.Width     Petal.Length    Petal.Width
 Min.   :4.300   Min.   :2.000   Min.   :1.000   Min.   :0.100
 1st Qu.:5.100   1st Qu.:2.800   1st Qu.:1.600   1st Qu.:0.300
 Median :5.800   Median :3.000   Median :4.350   Median :1.300
 Mean   :5.843   Mean   :3.057   Mean   :3.758   Mean   :1.199
 3rd Qu.:6.400   3rd Qu.:3.300   3rd Qu.:5.100   3rd Qu.:1.800
 Max.   :7.900   Max.   :4.400   Max.   :6.900   Max.   :2.500
       Species
```

Medidas sumárias/ descritivas

Para além de usarmos as função `summary()`, podemos usar funções específicas, `mean()`, `median()`, `sd()`, ...

```
> mean(iris$Sepal.Length)
[1] 5.843333
> median(iris$Sepal.Length)
[1] 5.8
> sd(iris$Sepal.Length)
[1] 0.8280661
> # coeficiente de  variação
> CV=sd(iris$Sepal.Length)/mean(iris$Sepal.Length)
> CV
[1] 0.1417113
```

Medidas sumárias/ descritivas

Podemos usar igualmente a função `summarize()` da livraria `dplyr`

```
> library(dplyr)
> summarize(iris, media=mean(Sepal.Length),
+           mediana=median(Sepal.Length),
+           desvi_padrao=sd(Sepal.Length),
+           cv=sd(Sepal.Length)/mean(Sepal.Length))
     media mediana desvi_padrao        cv
1 5.843333     5.8    0.8280661 0.1417113
>
> # alternativamente
> iris %>% summarize( media=mean(Sepal.Length),
+           mediana=median(Sepal.Length),
+           desvi_padrao=sd(Sepal.Length),
+           cv=sd(Sepal.Length)/mean(Sepal.Length))
     media mediana desvi_padrao        cv
1 5.843333     5.8    0.8280661 0.1417113
```

Medidas sumárias/ descritivas

A base de dados tem flores por espécies. Por exemplo, podemos sumarizar os dados por espécie.

```
> iris %>% group_by(Species) %>% summarize( media=mean(Sepal.Length),
+           mediana=median(Sepal.Length),
+           desvi_padrao=sd(Sepal.Length),
+           cv=sd(Sepal.Length)/mean(Sepal.Length))
# A tibble: 3 x 5
  Species    media mediana desvi_padrao     cv
  <fct>      <dbl>   <dbl>        <dbl>  <dbl>
1 setosa      5.01     5          0.352 0.0704
2 versicolor  5.94     5.9        0.516 0.0870
3 virginica   6.59     6.5        0.636 0.0965
```

Dados omissos/missing cases

A manipulação dos dados omissos será feita sobre a base de dados `iris` com dados omissos introduzidos propositadamente.

```
> iris_miss <- read.csv('iris_missing.csv',header = TRUE)
```

- Os dados omissos, no R, são denotados por `NA`.
- Pode-se usar a função `is.na()` para verificar a presença de dados omissos.
- `na.omit()` retorna um objecto/base de dados sem dados omissos.
- `complete.cases()` retorna um vector lógico indicando quais casos estão completos.

Dados omissos/missing cases

- Vamos examinar quantos dados omissos temos na base de dados.
- A contagem pode se fazer usando a função `colSums()`.

```
> colSums(is.na(iris_miss))
           X Sepal.Length  Sepal.Width Petal.Length  Petal.Width      Species
           0           15           15           15           15           15
```

- Podemos remover os dados omissos, apagando todas as linhas com `NA`.

```
> iris_miss <- read.csv('iris_missing.csv',header = TRUE)
> iris_no_miss <- na.omit(iris_miss)
> colSums(is.na(iris_no_miss))
           X Sepal.Length  Sepal.Width Petal.Length  Petal.Width      Species
           0            0            0            0            0            0
```

Dados omissos/missing cases

Por vezes há interesse de ver quais linhas/observações apresentam valores omissos.

```
> iris_miss[!complete.cases(iris_miss),]
      X Sepal.Length Sepal.Width Petal.Length Petal.Width    Species
1     1           NA         3.5          1.4         0.2     setosa
2     2          4.9         3.0          1.4         0.2       <NA>
6     6           NA         3.9          1.7         0.4     setosa
7     7          4.6         3.4           NA          NA     setosa
11   11          5.4         3.7           NA         0.2     setosa
13   13          4.8          NA          1.4         0.1     setosa
17   17          5.4         3.9          1.3         0.4       <NA>
20   20           NA         3.8           NA         0.3     setosa
22   22           NA         3.7          1.5         0.4     setosa
25   25           NA         3.4          1.9         0.2     setosa
28   28           NA         3.5          1.5         0.2     setosa
29   29          5.2          NA          1.4         0.2     setosa
32   32          5.4          NA          1.5          NA     setosa
37   37          5.5          NA           NA         0.2     setosa
38   38           NA         3.6          1.4         0.1     setosa
40   40          5.1         3.4          1.5          NA     setosa
42   42          4.5         2.3           NA         0.3     setosa
44   44          5.0         3.5          1.6         0.6       <NA>
59   59           NA         2.9           NA         1.3 versicolor
60   60          5.2         2.7           NA         1.4 versicolor
61   61          5.0         2.0           NA         1.0 versicolor
62   62           NA         3.0          4.2         1.5 versicolor
65   65          5.6         2.9          3.6         1.3       <NA>
```

Dados omissos/missing cases

A instrução `!complete.case()` nos da todas as linhas com valores omissos para todas as variáveis da base de dados.

Como fazer, se o interesse for verificar a presença de valores omissos em uma única variável? Acho que a função `is.na()` pode ajudar.

```
> is.na(iris_miss$Sepal.Length) # retorna um vector de TRUE ou FALSE  indicando se um dado valor é ou não
  [1]  TRUE FALSE FALSE FALSE FALSE  TRUE FALSE FALSE FALSE FALSE FALSE FALSE
 [13] FALSE FALSE FALSE FALSE FALSE FALSE FALSE  TRUE FALSE  TRUE FALSE FALSE
 [25]  TRUE FALSE FALSE  TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
 [37] FALSE  TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
 [49] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE  TRUE FALSE
 [61] FALSE  TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
 [73] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
 [85] FALSE FALSE FALSE FALSE FALSE  TRUE FALSE FALSE FALSE FALSE  TRUE FALSE
 [97] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE  TRUE FALSE FALSE FALSE
[109] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[121]  TRUE  TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[133] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[145] FALSE FALSE  TRUE FALSE FALSE FALSE
```

Dados omissos/missing cases

Para saber a posição vamos usar a função `which()`.

```
> pos <- which(is.na(iris_miss$Sepal.Length))
> iris_miss[pos, ]
      X Sepal.Length Sepal.Width Petal.Length Petal.Width    Species
1     1           NA         3.5          1.4         0.2     setosa
6     6           NA         3.9          1.7         0.4     setosa
20   20           NA         3.8           NA         0.3     setosa
22   22           NA         3.7          1.5         0.4     setosa
25   25           NA         3.4          1.9         0.2     setosa
28   28           NA         3.5          1.5         0.2     setosa
38   38           NA         3.6          1.4         0.1     setosa
59   59           NA         2.9           NA         1.3 versicolor
62   62           NA         3.0          4.2         1.5 versicolor
90   90           NA          NA          4.0         1.3 versicolor
95   95           NA         2.7          4.2         1.3       <NA>
105 105           NA         3.0          5.8         2.2  virginica
121 121           NA         3.2          5.7          NA  virginica
122 122           NA         2.8          4.9         2.0  virginica
147 147           NA          NA          5.0         1.9  virginica
```

Os valores omissos poder ser representados por diversos caracteres: 9999, *, ? , " ", ...

Como informar o R que estes caracteres representam valores omissos?

- Durante a leitura dos dados.

```
> iris_spec <- read.csv('iris_missing_spec_car.csv',header=TRUE, sep=',' , na.strings = "9999")
> colSums(is.na(iris_spec))
Sepal.Length  Sepal.Width Petal.Length  Petal.Width      Species
          15           15           15           15           15
```

- Após a leitura,

```
> iris_spec1 <- read.csv('iris_missing_spec_car.csv',header=TRUE, sep=',')
> iris_spec1[iris_spec1 == 9999] <- NA
> colSums(is.na(iris_spec1))
Sepal.Length  Sepal.Width Petal.Length  Petal.Width      Species
          15           15           15           15           15
```

Dados omissos/missing cases vs estatísticas descritivas

```
> summary(iris_spec)
  Sepal.Length    Sepal.Width     Petal.Length    Petal.Width
 Min.   :4.300   Min.   :2.000   Min.   :1.000   Min.   :0.10
 1st Qu.:5.100   1st Qu.:2.800   1st Qu.:1.600   1st Qu.:0.30
 Median :5.800   Median :3.000   Median :4.400   Median :1.30
 Mean   :5.867   Mean   :3.072   Mean   :3.764   Mean   :1.16
 3rd Qu.:6.400   3rd Qu.:3.350   3rd Qu.:5.100   3rd Qu.:1.80
 Max.   :7.900   Max.   :4.400   Max.   :6.900   Max.   :2.50
 NA's   :15      NA's   :15      NA's   :15      NA's   :15
   Species
 Length:150
 Class :character
 Mode  :character
```

```
> mean(iris_spec$Sepal.Length,na.rm = TRUE)
[1] 5.866667
```

Tabelas de frequência

Para variáveis de natureza qualitativa (não-numéricas) não se pode calcular as estatísticas sumárias. Mas pode-se sumarizar através de contagens ou percentagens.

- Para construir uma tabela de frequência vamos usar a função `table()`.

```
> table(iris$Species)

    setosa versicolor  virginica
        50         50         50
```

Para ter dados em percentagem vamos a usar a função `prop.table()`

```
> prop.table(table(iris$Species))

    setosa versicolor  virginica
 0.3333333  0.3333333  0.3333333
```

Tabelas de frequência

A livraria `dplyr` também permite fazer tabelas de frequência e ter os resultados num formato 'agradável'.

```
> library(dplyr) # não se esqueça de chamar esta livraria
> iris%>% group_by(Species)%>%
+   summarize (N = n())%>% mutate(Perc = N/sum(N))
# A tibble: 3 x 3
  Species        N  Perc
  <fct>      <int> <dbl>
1 setosa        50 0.333
2 versicolor    50 0.333
3 virginica     50 0.333
```

Visualização de dados - Gráficos

O maior valor de uma imagem é quando ela nos força a perceber o que nunca esperávamos ver. — John Tukey

Com o incremento contínuo de dados, é impossível contar histórias sem o auxílio de gráficos.

Como escolher o gráfico ideal

Gráficos

- O R é bastante apreciado devido a sua capacidade de fazer gráficos elegantes.
- Com o R pode-se obter qualquer tipo de gráfico.
- Gráficos produzidos em R podem ser gravados em vários formatos, pdf,png, jpg, ...

Gráficos

- O R tem algumas funções predefinidas que podem ser usadas para uma rápida exploração de dados.
- `plot()` geralmente usada para diagrama de dispersão.
- `barplot()` gráfico de barras.
- `hist()` histograma.
- `boxplot()` caixa de bigodes.

Grafico de barras

Vamos mostrar como pode-se construir um gráfico de barras usando a base de dados `mtcars`.

```
> data("mtcars")
> freq <- table(mtcars$gear)
> barplot(freq)
```

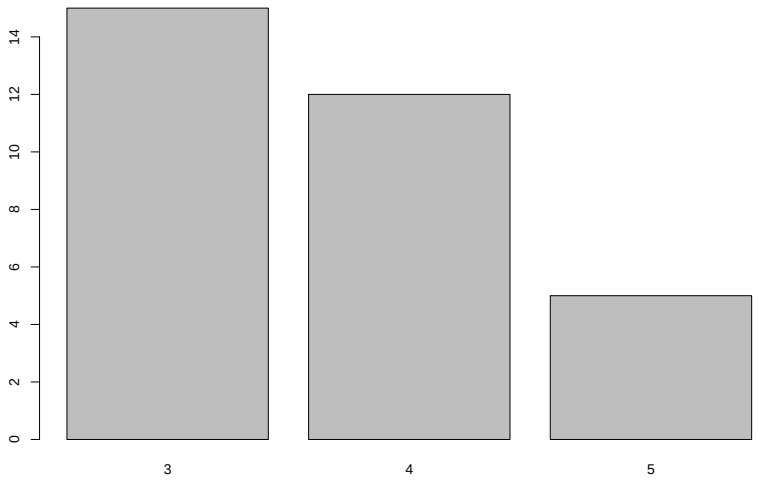


Grafico de barras

A função `barplot` tem vários argumentos que nos permite criar um gráfico com as características desejadas.

- `col` : escolher a cor das barras;
- `names.arg`: Um vector de nomes a serem exibidos abaixo de cada barra ou grupo de barras.
- `legend.text`: vector de texto para costruir a lengeda do gráfico.
- `horiz`: Valor lógico indicando se as barras devem ser verticais ou horizontais.

Grafico de barras

```
> data("mtcars")
> freq <- table(mtcars$am,mtcars$gear)
> freq

   3  4  5
0 15  4  0
1  0  8  5
> barplot(freq,beside = TRUE, col=c('blue','red'),legend.text = c('Automático', 'Manual'))
```

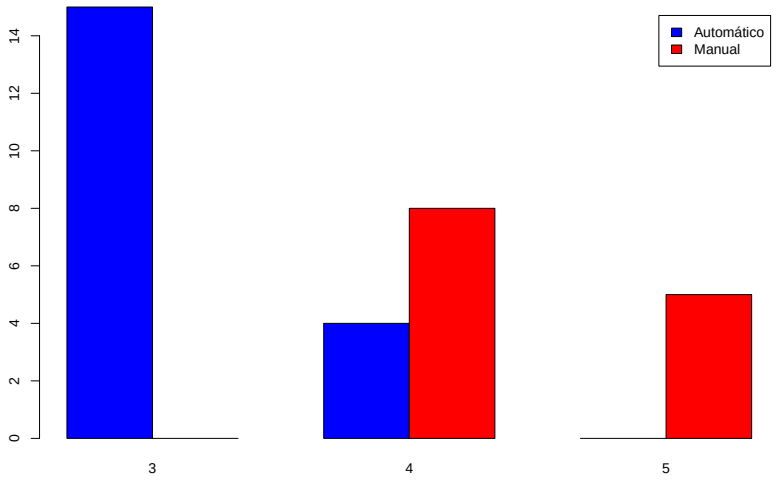
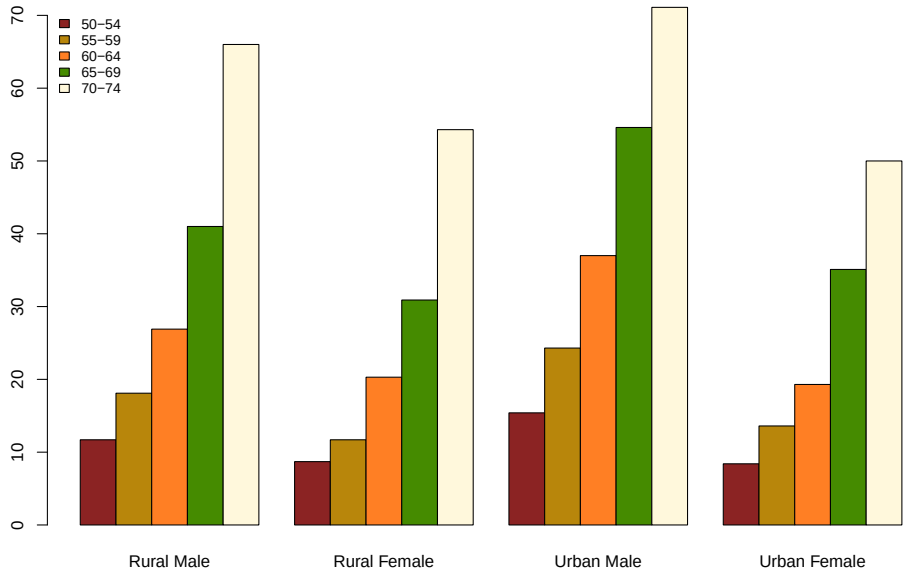


Grafico de barras

```
> data("VADeaths")
> cores <- c("brown4", "darkgoldenrod", "chocolate1","chartreuse4", "cornsilk")# definir as cores
> barplot(VADeaths, col = cores, beside = TRUE) # gráfico de barras
> # Acrescentar legenda
> legend("topleft", legend = rownames(VADeaths),
+        fill = cores, box.lty = 0, cex = 0.8) # Acrescentar legenda
```

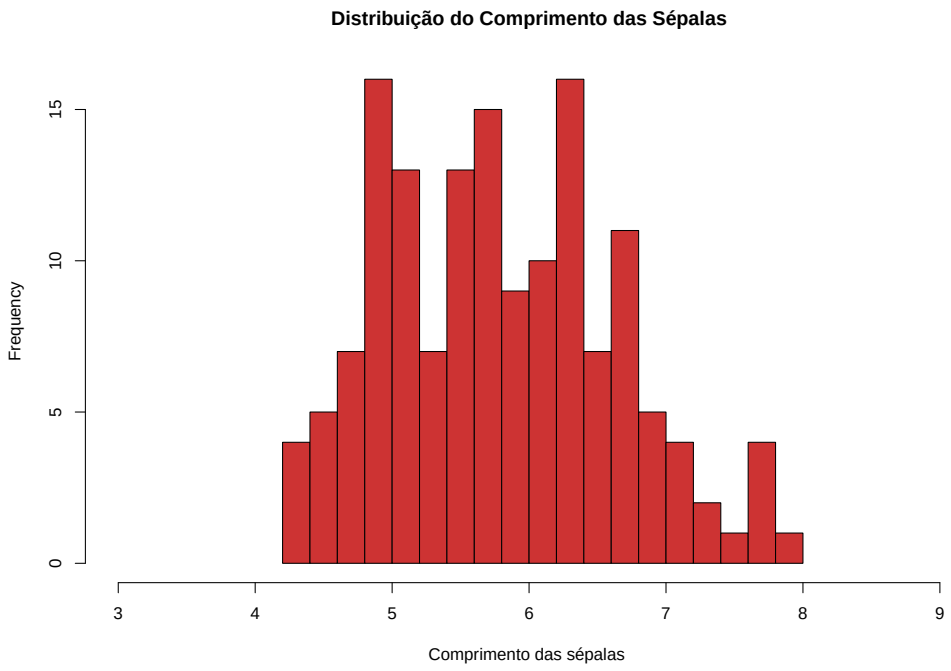


Histograma

- Este é um gráfico para variáveis de natureza contínua (numéricas).
- Serve para verificar a distribuição dos dados (Simétricos/Assimétricos)
- Permite identificar observações atípicas (outliers)
- Pode-se construir usando a função `hist()`.

Histograma

```
> hist(iris$Sepal.Length, col ='brown3',breaks = 20,xlim = c(3, 9),
+      xlab = 'Comprimento das sépalas ', main='Distribuição do Comprimento das Sépalas')
```

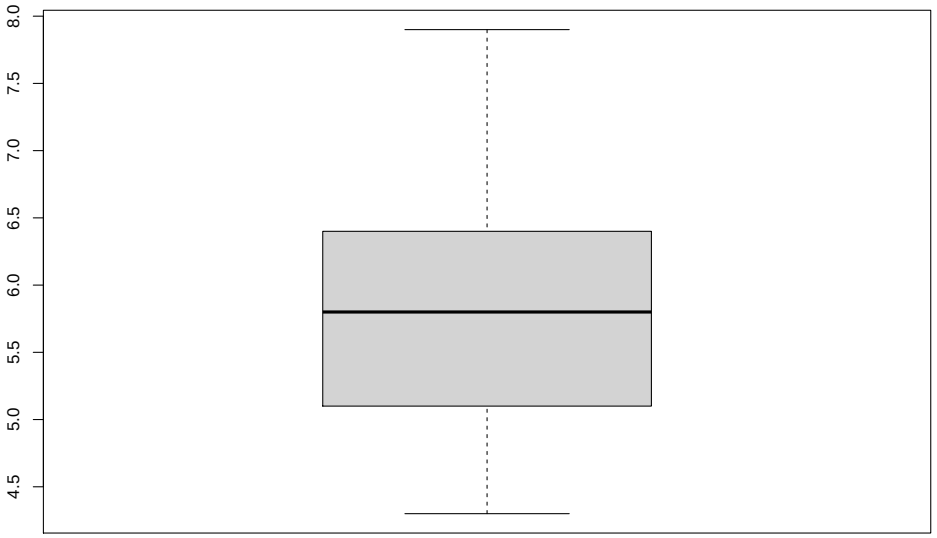


Box-plot

- Box-plot também é um gráfico concebido para variáveis contínuas.
- Permite conhecer a distribuição dos dados
- Identificar observações atípicas.
- Tem a vantagem de permitir comparações.
- Pode-se construir usando a função `boxplot()`

Box-plot

```
> boxplot(iris$Sepal.Length)
```



Box-plot

Vamos comparar o comprimento das sépalas entre as espécies.

```
> boxplot(iris$Sepal.Length~iris$Species,
+         xlab='Espécie', ylab='Comprimento da Sépala',col='orange2')
```

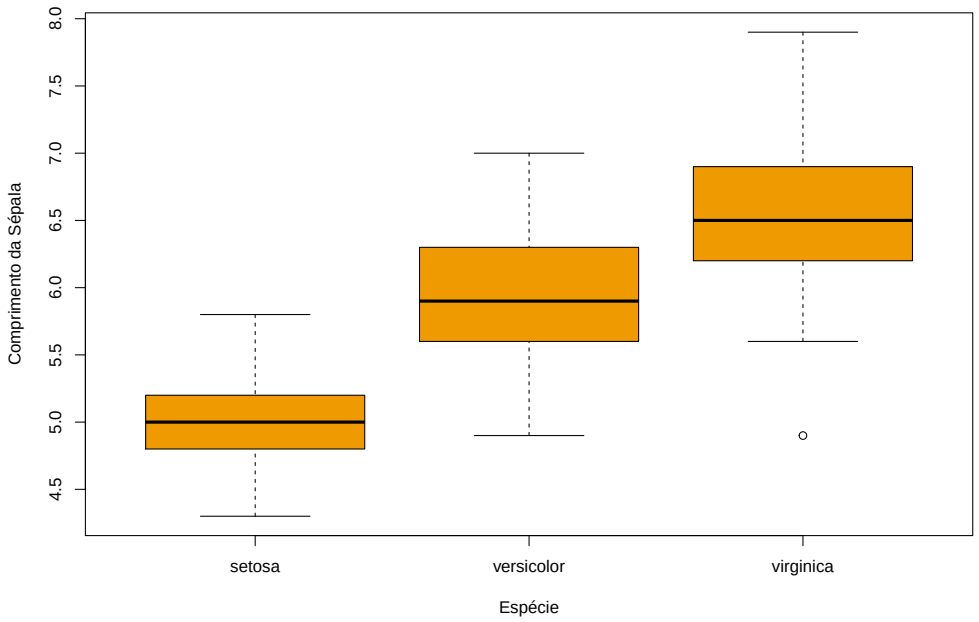


Diagrama de dispersão

- O diagrama de dispersão serve para mostrar a relação entre duas variáveis numéricas.

- Para a sua concepção vamos usar a função `plot()`.

```
> plot(iris$Sepal.Length,iris$Petal.Width)
```

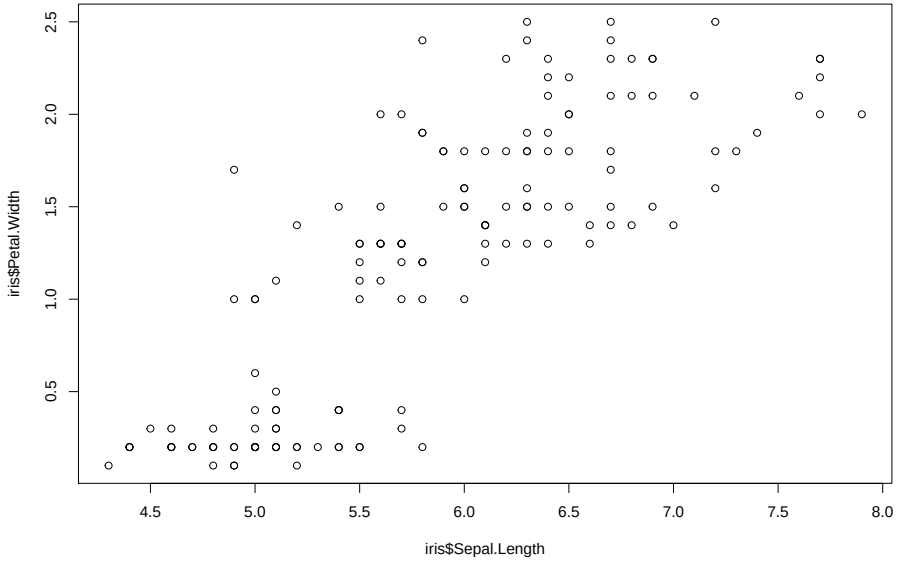
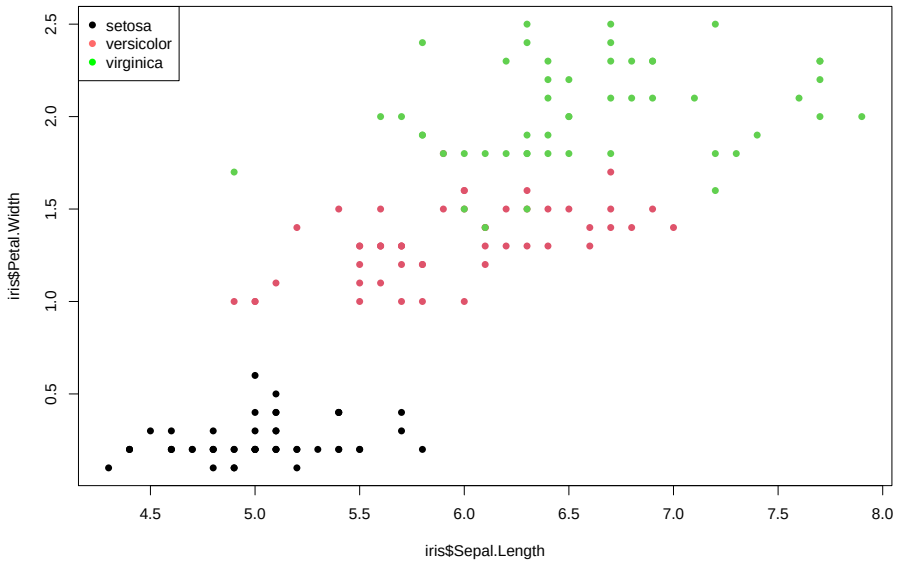


Diagrama de dispersão

A relação entre o comprimento das sépalas e a largura das pétalas pode ser analisada para as três espécies.

```
> plot(iris$Sepal.Length,iris$Petal.Width, col=iris$Species,pch = 16)
> legend('topleft', legend =c('setosa','versicolor','virginica'),
+        col =c('black','indianred1','green'),pch=16)
```



Análise bivariada

- No R, podemos também construir tabelas cruzadas
- Verificar relações entre duas variáveis , qualitativas/ quantitativas
- Para construir tabelas cruzadas temos várias opções:
  - `table()`.
  - `crosstab()` e `CrossTable()` da livraria `descr`.
  - `CrossTable()` da livraria `gmodels`.

Análise bivariada- Tabelas cruzadas

```
> library(haven)
> dados_pesca <- read_spss('BaseDados.sav')
> # tipo de pesca vs metodo de conservação
> table(dados_pesca$PTP, dados_pesca$MC)

     1   2   3   4   5   6
  1 53  11  41 384 120   6
  2 28  12  70  21 119  12
  3 37  11  24   6  19  16
  4  7   7   1   9   5   3
  5 76   6  39  66  30  46
  9 145  3  26  50  20  31
```

Análise bivariada- Tabelas cruzadas

```
> library(descr)
> # tipo de pesca vs metodo de conservação
> crosstab(dados_pesca$PTP,dados_pesca$MC, plot=FALSE)
   Cell Contents
|-------------------------|
|                   Count |
|-------------------------|


=============================================================================
                                 Metodo para conservar e/ou processar pescado
prncpl tp d psc q a s fml prt       1      2      3      4      5      6   Total
-----------------------------------------------------------------------------
1                                  53     11     41    384    120      6     615
-----------------------------------------------------------------------------
2                                  28     12     70     21    119     12     262
-----------------------------------------------------------------------------
3                                  37     11     24      6     19     16     113
-----------------------------------------------------------------------------
4                                   7      7      1      9      5      3      32
-----------------------------------------------------------------------------
5                                  76      6     39     66     30     46     263
-----------------------------------------------------------------------------
```

Análise bivariada- Tabelas cruzadas

Como verificar associação entre duas várias

- `chisq.test()`.
- `crosstab` e especificar o argumento `chisq=TRUE`.

```
> library(descr)
> # tipo de pesca vs metodo de conservação
> chisq.test(dados_pesca$PTP,dados_pesca$MC)
Warning in chisq.test(dados_pesca$PTP, dados_pesca$MC): Chi-squared
approximation may be incorrect

    Pearson's Chi-squared test

data:  dados_pesca$PTP and dados_pesca$MC
X-squared = 776.21, df = 25, p-value < 2.2e-16
```

Análise bivariada- Variáveis quantitativas

Para variáveis quantitativas a análise é feita por via:

- Diagrama de dispersão.
- Análise das correlações, calculado usando a função `cor()`.
- Análise de regressão, feita a usando a função `lm()`.

Análise bivariada- Variáveis quantitativas

calculo do coeficiente de correlação de pearson

```
> dados_carros <- mtcars[, c(1,3,4,5,6,7)]
> # calcular o coeficiente de correlação de pearson para duas
> cor(dados_carros$mpg,dados_carros$disp)
[1] -0.8475514
> # matriz das correlações
> cor(dados_carros)
            mpg       disp         hp        drat         wt        qsec
mpg   1.0000000 -0.8475514 -0.7761684  0.68117191 -0.8676594  0.41868403
disp -0.8475514  1.0000000  0.7909486 -0.71021393  0.8879799 -0.43369788
hp   -0.7761684  0.7909486  1.0000000 -0.44875912  0.6587479 -0.70822339
drat  0.6811719 -0.7102139 -0.4487591  1.00000000 -0.7124406  0.09120476
wt   -0.8676594  0.8879799  0.6587479 -0.71244065  1.0000000 -0.17471588
qsec  0.4186840 -0.4336979 -0.7082234  0.09120476 -0.1747159  1.00000000
```

Análise bivariada- Variáveis quantitativas

Regressão linear

```
> # regressão linear simples
> lm(mpg~disp,data = dados_carros)

Call:
lm(formula = mpg ~ disp, data = dados_carros)

Coefficients:
(Intercept)         disp
   29.59985     -0.04122
> #ou
> reg<- lm(mpg~disp,data = dados_carros)
> summary(reg)

Call:
lm(formula = mpg ~ disp, data = dados_carros)

Residuals:
    Min      1Q  Median      3Q     Max
-4.8922 -2.2022 -0.9631  1.6272  7.2305
```

Ciclos e Estruturas de control

Funções

- Um dos aspectos bastante atractivo do R é a facilidade de poder escrever funções próprias;
- Funções podem receber um ou mais argumentos e devolvem um único valor.

```
Fun1 = function(x){
  return(x*x)
}
```

A função `Fun1` recebe um valor `x` e devolve o quadrado desse valor.

Os elementos chaves na definição de uma função são:

- A palavra chave `function`;
- Os parenteses `()` para acomodar os argumentos da função;
- Sequência de operações dentro de chavetas `{}`;
- o comando `return`;

```
Fun1(5) ; Fun1(10)
```

```
## [1] 25
```

```
## [1] 100
```

A função a seguir recebe um vector, e devolve a média e o desvio padra

```
media_var = function(values){
  return(c(mean(values),sd(values)))
}


w = rep(c(1,3,4), each =5)
media_var(w)
```

```
## [1] 2.666667 1.290994
```

Escopo da variável

Quando se envoca uma função, o R primeiro procura por variáveis locais, e depos por variáveis globais. Por exemplo, a função `Fun4` usa uma variável global.

```
blob = 5
Fun4 = function(){
    return(blob)
 }
Fun4()
```

```
## [1] 5
```

Geralmente, as varíaveis/objectos devem ser definidas localmente.

Esruturas de control- if

As instruções condicionais são recursos de uma linguagem de programação que executam cálculos ou acções diferentes, dependendo de uma condição ser avaliada como VERDADEIRA (TRUE) ou FALSA (FALSE). Elas são usadas em quase todos os programas de computador.

Esruturas de control- if

A declaração `if` tem a seguinte estrutura básica:

```
if(expr){
  ## executar alguma tarefa
}
```

Onde `expr` é avaliado se é verdadeiro ou falso.

Esruturas de control- if

O exemplo a seguir mostra o funcionamento de uma estrutura de control `if`.

```
x = 5
y = 5
if(x<5) {
 y = 0
}
y
```

```
## [1] 5
```

Nesse pedaço de código, x <5 é avaliado como FALSE, portanto os seguintes colchetes não são avaliados.

if-esle

Podemos interligar uma série de instruções `if`

```
x = 0
if(x > 0) {
 cat("x é maior que zero")
 } else if(x < 0) {
 cat("x é meno que zeo")
 } else {
 cat("x dever ser zero!")
 cat("\n")
 }
```

```
## x dever ser zero!
```

if-else

```
IsNegative = function(value) {
 is_pos = FALSE
 if(value < 0) {
 is_pos = TRUE
 }
 return(is_pos)
}

IsNegative(1)
```

```
## [1] FALSE
```

```
IsNegative(-5.6)
```

```
## [1] TRUE
```

if-else

```
IsGreaterThan = function(value1, value2) {
 is_greater_than = FALSE
 if(value1 > value2) {
 is_greater_than = TRUE
 }
 return(is_greater_than)
}
IsGreaterThan(-5, -6)
```

```
## [1] TRUE
```

Ciclos-for

- Suponha que deseja calcular $\sum_{i=1}^{10} i^2$;
- Em R pode-se usar um ciclo. Permite fazer uma certa instrução repetidas vezes.

```
x = 0
for(i in 1:10) {
 x = x+i^2
}
x
```

```
## [1] 385
```

Ciclos-for

- $\sum_{i=-5}^{-1} e^i/i$

```
total = 0
for(i in -5:-1) {
 total = total + exp(i)/i^2
 }
total
```

```
## [1] 0.4086594
```

Ciclos-for

- Calcule $\sum e^k/k$, $k = 3, 6, 9 \ldots, 21$

```
total = 0
for(i in 1:7) {
 k = i*3
 total = total + exp(k)/k^2
 }
 total
```

```
## [1] 3208939
```

Função apply()

- Usamos a função `apply()`quando queremos aplicar a mesma função a todas as linhas ou colunas de um quadro de dados.
- Suponha que temos uma data frame com três colunas.

```
(df4 = data.frame(c1 = 1:4, c2 = 4:7, c3 = 2:5))
```

```
##   c1 c2 c3
## 1  1  4  2
## 2  2  5  3
## 3  3  6  4
## 4  4  7  5
```

Desvio-padrão para cada coluna.

```
apply(df4, 2, sd)
```

```
##        c1        c2        c3
## 1.290994 1.290994 1.290994
```

Suponha que uma das colunas não é numérica.

```
(df5 = data.frame(c1 = 1:3, c2 = 4:6, c3 = LETTERS[1:3]))
```

```
##   c1 c2 c3
## 1  1  4  A
## 2  2  5  B
## 3  3  6  C
```

O cálculo da média não faz sentido

```
apply(df5, 1, mean)
```

```
## [1] NA NA NA
```

O ideal é remover a coluna que não é numérica

```
apply(df5[ ,1:2], 1, mean)
```

```
## [1] 2.5 3.5 4.5
```

Testes de Hipótese

Testes de hipotese

- Os testes paramétricos assumem que os dados são gerados a partir de uma certa família de distribuições de probabilidade (geralmente distribuições normais);
- Os testes não paramétricos não presumem que os dados vêm de qualquer família específica de distribuições de probabilidade;
- Os testes paramétricos são normalmente mais fáceis de calcular e mais poderosos quando suas suposições são atendidas;

-Em caso de dúvida, provavelmente é mais seguro usar testes não paramétricos;

Teste t para uma amostra

- Um teste t de uma amostra testa se uma sequência de amostras de valor real vem de uma distribuição com uma média (média da população) igual a algum fixo μ_0.
- O teste t é um teste paramétrico: ele assume que os dados têm uma distribuição normal.
- O comprimento médio das sépalas é 7 cm.

```
> t.test(iris$Sepal.Length, mu=7)


    One Sample t-test


data:  iris$Sepal.Length
t = -17.108, df = 149, p-value < 2.2e-16
alternative hypothesis: true mean is not equal to 7
95 percent confidence interval:
 5.709732 5.976934
```

FicaEmCasa

Teste de Wilcoxon

- Testa se uma sequência de amostras com valor real vem de uma distribuição com uma mediana igual a μ_0.
- É mais robusto, mas menos poderoso do que o teste t

```
> wilcox.test(iris$Sepal.Length, mu=7)

    Wilcoxon signed rank test with continuity correction

data:  iris$Sepal.Length
V = 328, p-value < 2.2e-16
alternative hypothesis: true location is not equal to 7
```

Teste t para duas amostras independentes

- Um teste t de duas amostras testa se duas sequências de amostras de valor real vêm de distribuições com médias diferentes.

-Este é um teste paramétrico, que assume que ambas as sequências(amostras) são normalmente distribuídas.

- Exemplo: o comprimento da espécie setosa é maior que o da espécie virginica.

```
> iris1<- iris[iris$Species %in% c('setosa','virginica'),]
> iris1$Species<-as.character(iris1$Species)
> t.test(iris1$Sepal.Length~iris1$Species)

    Welch Two Sample t-test
```

FicaEmCasa

Teste t para amostras emparrelhadas

- Um teste t pareado é usado quando há duas medições em cada item.

-Exemplo: a rodovia milhas por galão é melhor do que a cidade milhas por galão?

```
> library(ggplot2)
> data("mpg")
> t.test(mpg$hwy,mpg$cty,paired=TRUE)

    Paired t-test

data:  mpg$hwy and mpg$cty
t = 44.492, df = 233, p-value < 2.2e-16
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 6.289765 6.872628
sample estimates:
```

Análise de Variância (ANOVA)

- Análise de Variância (ANOVA) é um procedimento utilizado para comparar, isto é, testar a igualdade de três ou mais médias populacionais (grupos, tratamentos, factor), baseado na análise das variâncias amostrais.
- Os dados amostrais são separados em grupos segundo uma ou mais características (factor(es)).
- Factor (ou tratamento) é uma característica que permite distinguir diferentes populações ou subgrupos.
- Cada factor contém duas ou mais classificações.

Pressupostos da ANOVA

Para realização duma ANOVA, os seguintes pressupostos devem ser verificados.

- Amostras são aleatórias e mutuamente independentes.
- Populações ou subgrupos tem a mesma variância. A distribuição da característica de interesse é normal em cada população ou subgrupo.

Hipoteses da ANOVA

$H0$: as médias de todas as populações são iguais, ou seja, o tratamento (factor) não tem efeito.

$H1$: nem todas a médias populacionais são iguais, ou seja, pelo menos uma média é diferente, isto é, existe efeito do tratamento (factor).

$H_0 : \mu_1 = \mu_2 = \cdots = \mu_r$

$H_1 : \mu_i \neq \mu_j$

Métodos avançados de leitura de dados

Métodos avançados de leitura de dados

Objectivos

Espera-se que no final da sessão:

- Saber ler tabelas HTML alojadas numa página web.
- Saber ler tabelas em ficheiros `pdf`.
- Saber ler dados no formato `xlsx`.

Métodos avançados de leitura de dados

- Muitas tabelas de dados nas páginas da web são exibidas usando um formato HTML.
- O R dispões de um pacote que facilita a leitura de dados, `rvest`
- O processo de leitura será ilustrado usando a informação climática disponível em http://www.infoplease.com/ipa/A0762183.html.

Métodos avançados de leitura de dados

```
> library(rvest)
> library(xml2)
> library(httr)
> sel='#tablepress-73' # onde a tabela está localizada na web  CSS selector.
> url_data <- "https://ourworldindata.org/famines"
> tabela <-url_data %>%
+   read_html() %>% # ler o conteudo da página
+   html_node(css =sel)%>% # ler algo especifico na página
+   html_table()
> head(tabela)
     Year    Country Excess Mortality midpoint Excess Mortality lower
1 1846-52    Ireland                 1,000,000              1,000,000
2  1860-1      India                 2,000,000              2,000,000
3 1863-67 Cape Verde                    30,000                 30,000
4  1866-7      India                   961,043                961,043
5    1868    Finland                   100,000                100,000
6 1868-70      India                 1,500,000              1,500,000
  Excess Mortality upper                                       Source
1              1,000,000                                Ó Gráda (2007)
2              2,000,000         Kumar and Raychaudhuri [Eds.] (1983)
3                 30,000                          Ó Gráda (2009), p. 22
4                 961,043         Kumar and Raychaudhuri [Eds.] (1983)
```

FicaEmCasa

Métodos avançados de leitura de dados

- O R possui pacotes que permitem ao usuário extrair tabelas de um ficheiro pdf.
- Para este exercício vamos usar o pacote `tabulizer`.
- Como exemplo vamos usar o relatório sobre o índice global da paz. este pode ser encontrado no seguinte endereço

Métodos avançados de leitura de dados

```
> library(tabulizer)
> library(tidyverse)
> data_indice <- extract_tables("Global-Peace-Index-2018-2.pdf", output = 'data.frame', pages=10)
> head(data_indice[[1]])
  RANK     COUNTRY SCORE CHANGE RANK.1 COUNTRY.1 SCORE.1 CHANGE.1 RANK.2
1    1     Iceland 1.096    ï\201¾     29  Botswana   1.659    ï\201£ 4     57
2    2 New Zealand 1.192    ï\201¾     30     Spain   1.678   ï\201£ 10     58
3    3     Austria 1.274  ï\201¢ 1     31    Latvia   1.689    ï\201¢ 1     59
4    4    Portugal 1.318  ï\201£ 1     32    Poland   1.727    ï\201¢ 1     60
5    5     Denmark 1.353    ï\201¾     33   Estonia   1.732    ï\201¢ 3     61
6    6      Canada 1.372    ï\201¾     34    Taiwan   1.736    ï\201¢ 3     62
       COUNTRY.2 SCORE.2 CHANGE.2
1 United Kingdom   1.876    ï\201£ 6
2     Montenegro   1.893    ï\201¢ 5
3    Timor-Leste   1.895    ï\201£ 5
4        Vietnam   1.905      ï\201¾
5         France   1.909    ï\201£ 5
6         Cyprus   1.913    ï\201¢ 3
```

Métodos avançados de leitura de dados

No processo de extracção podemos também especificar a área especifica onde extrair a tabela.

```
> # locate_areas(path,pages=10)  apara selecionar a area que pretendemos extrair da tabela
> data_indice<- extract_tables("Global-Peace-Index-2018-2.pdf",output = "data.frame",pages = c(10,10,10),
+                              area = list(c(498.52615,42.24403, 788.29295,171.41170),
+                                          c(492.6525, 210.5534 ,796.1245, 339.7211),
+                                          c(492.6525, 210.5534, 796.1245,339.7211)),
+                                 guess = FALSE)
> tabela_final<-do.call(rbind,data_indice)
> head(tabela_final)
  RANK     COUNTRY SCORE
1    1     Iceland 1.096
2    2 New Zealand 1.192
3    3     Austria 1.274
4    4    Portugal 1.318
5    5     Denmark 1.353
6    6      Canada 1.372
```

Métodos avançados de leitura de dados

Podemos ler dados em excel com extensão `.xlsx`.

```
> library(readxl)
> dados_excl<-read_excel("Dados_semanais.xlsx", sheet='Dados_semanais')
> head(dados_excl)
# A tibble: 6 x 7
   ...1 distrito time                 tas tasmax tasmin     pr
  <dbl> <chr>    <dttm>             <dbl>  <dbl>  <dbl>  <dbl>
1     1 ANGOCHE  1989-01-01 00:00:00  27.1   28.6   25.0 0.0636
2     2 ANGOCHE  1989-01-08 00:00:00  27.1   28.5   24.9 0.109
3     3 ANGOCHE  1989-01-15 00:00:00  25.7   27.4   24.7 0.353
4     4 ANGOCHE  1989-01-22 00:00:00  27.1   29.5   24.9 0.0306
5     5 ANGOCHE  1989-01-29 00:00:00  26.0   27.0   24.4 0.187
6     6 ANGOCHE  1989-02-05 00:00:00  27.2   28.8   25.7 0.0865
```